

# Constitutional Discovery Report — CDR-003

## Event Bus Consolidation

**CEP:** 003 **Status:** Discovery Complete — awaiting approval **CEP Objective:** Eliminate duplicate runtime ownership of the event bus. 3 event buses → 1 canonical. Satisfies Canonical Ownership Law §10, Simplicity Rule §4, Universal Event Law §14. **EX-02 Compliance:** Infrastructure topology included. Constitutional Evidence Boundary declared.

### 1. Constitutional Evidence Boundary

| Scope                 | Detail                                                                                                                              |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Audited               | Frontend event bus systems: <code>runtimes/event-bus.ts</code> , <code>api/event-bus.ts</code> , <code>lib/event-bus.ts</code>      |
| Not audited           | Backend event systems, SSE endpoints, polling loops (CEP-006 scope), Workspace Runtime events                                       |
| Proven                | <code>lib/event-bus.ts</code> has zero real imports (confirmed via static analysis, dynamic import check, registry check)           |
| Supported by evidence | <code>api/event-bus.ts</code> has 3 consumers (use-ai-presence, use-query, use-realtime-sync) — all confirmed via import resolution |
| Inferred              | Migration pattern (change import + rename method) is constitutional — no new abstractions needed                                    |
| Unknown               | None within CEP scope                                                                                                               |

### 2. Infrastructure Topology

#### 2.1 Event Bus 1: `runtimes/event-bus.ts` (CANONICAL)

| Property    | Value                                                                                    |
|-------------|------------------------------------------------------------------------------------------|
| File        | <code>frontend/src/runtimes/event-bus.ts</code>                                          |
| Lines       | 79                                                                                       |
| Exports     | <code>RuntimeEvent</code> (type), <code>bus</code> (object)                              |
| Methods     | <code>emit(type, payload)</code> , <code>on(type, handler)</code> , <code>clear()</code> |
| Event types | <code>RuntimeEvent</code> union type                                                     |

#### Publishers (emit):

| Consumer                                     | Events Published              |
|----------------------------------------------|-------------------------------|
| <code>runtimes/commitment/engine.ts</code>   | Commitment lifecycle events   |
| <code>runtimes/conversation/engine.ts</code> | Conversation lifecycle events |
| <code>runtimes/experience/engine.ts</code>   | Executive context changes     |
| <code>runtimes/graph/engine.ts</code>        | Graph relationship events     |
| <code>runtimes/intelligence/engine.ts</code> | Intelligence insight events   |
| <code>runtimes/object/engine.ts</code>       | Object lifecycle events       |
| <code>runtimes/orchestrator.ts</code>        | Runtime lifecycle events      |
| <code>runtimes/state-fabric.ts</code>        | State snapshot events         |
| <code>runtimes/timeline/engine.ts</code>     | Timeline events               |
| <code>runtimes/workspace/store.ts</code>     | Workspace state changes       |

#### Subscribers (on):

| Consumer                                 | Subscribes To                                                                        |
|------------------------------------------|--------------------------------------------------------------------------------------|
| <code>runtimes/workspace/store.ts</code> | <code>ObjectLoaded</code> , <code>TimelineLoaded</code> , <code>ObjectUpdated</code> |

**Event flow:** Typed event system. Events are `RuntimeEvent` union — compiler-enforced.

**Event ownership:** Canonical. Single source of truth for runtime events.

**Duplicate emissions:** None. Each event emitted by exactly one publisher.

**Failure propagation:** No explicit failure handling — events are fire-and-forget. No retry.

**Synchronous boundaries:** All events are synchronous (no async event loop). No queuing.

**Asynchronous boundaries:** None.

#### 2.2 Event Bus 2: `api/event-bus.ts` (LEGACY)

| Property    | Value                                                             |
|-------------|-------------------------------------------------------------------|
| File        | <code>frontend/src/api/event-bus.ts</code>                        |
| Lines       | 23                                                                |
| Exports     | <code>eventBus</code> (object)                                    |
| Methods     | <code>emit(type, payload)</code> , <code>on(type, handler)</code> |
| Event types | Untyped string-based events                                       |

#### Publishers (emit):

| Consumer                              | Events Published                                                                |
|---------------------------------------|---------------------------------------------------------------------------------|
| <code>api/use-ai-presence.ts</code>   | <code>'ai:insight'</code> (via <code>eventBus.emit('ai:insight', best)</code> ) |
| <code>api/use-query.ts</code>         | Generic data refresh events (via <code>eventBus.emit(...)</code> )              |
| <code>api/use-realtime-sync.ts</code> | Delta sync events (via <code>eventBus.emit(...)</code> )                        |

#### Subscribers (on):

| Consumer                      | Subscribes To                                       |
|-------------------------------|-----------------------------------------------------|
| <code>api/use-query.ts</code> | Refresh events (via <code>eventBus.on(...)</code> ) |

**Event flow:** String-based, untyped. No compile-time event type checking.

**Event ownership:** Orphaned — no clear owner. Events are ad-hoc strings.

**Duplicate emissions:** `use-ai-presence` deduplicates via `sessionStorage`, but the event bus itself has no deduplication.

**Failure propagation:** No failure handling. Fire-and-forget with try/catch in consumers.

**Synchronous boundaries:** Synchronous.

**Asynchronous boundaries:** None.

#### 2.3 Event Bus 3: `lib/event-bus.ts` (DEAD)

| Property | Value |
|----------|-------|
|----------|-------|

|           |                                                              |
|-----------|--------------------------------------------------------------|
| File      | frontend/src/lib/event-bus.ts                                |
| Lines     | 46                                                           |
| Methods   | on, emit, onAny, off, listenerCount                          |
| Consumers | <b>ZERO</b> — confirmed dead. No real imports from any file. |

**Deletion justification:** Zero real imports. Zero dynamic references. Zero registry lookups. File is 1.2KB of dead code.

3. Canonical Owner Confirmation

| Role                | Owner                 | Evidence                                                                                          |
|---------------------|-----------------------|---------------------------------------------------------------------------------------------------|
| Canonical Event Bus | runtimes/event-bus.ts | 14 consumers, including all runtimes. Typed event system. Integrated with orchestrator lifecycle. |
| Legacy              | api/event-bus.ts      | 3 consumers (use-ai-presence, use-query, use-realtime-sync). All also import canonical bus.       |
| Dead                | lib/event-bus.ts      | 0 consumers. Zero real imports.                                                                   |

4. Consumer Dependency Graph

```
api/event-bus.ts (LEGACY – to be removed)
├─ api/use-ai-presence.ts      (imports eventBus, calls eventBus.emit('ai:insight', ...))
├─ api/use-query.ts           (imports eventBus, calls eventBus.emit() + eventBus.on())
└─ api/use-realtime-sync.ts   (imports eventBus, calls eventBus.emit())

lib/event-bus.ts (DEAD – to be removed immediately)
└─ Zero consumers – confirmed

runtimes/event-bus.ts (CANONICAL – remains)
└─ 14 consumers (all runtimes + 3 API hooks that also import this)
```

**Key insight:** The 3 API hooks already import BOTH event buses. They import `eventBus` from `./event-bus` (legacy) AND `bus` from `../runtimes/event-bus` (canonical). The migration is: drop the legacy import, use the canonical import.

5. Migration Sequence

```
Step 1: Remove lib/event-bus.ts (dead – zero consumers, safe immediately)
Step 2: Migrate api/use-ai-presence.ts
  - Change: import { eventBus } from './event-bus' → import { bus } from '../runtimes/event-bus'
  - Change: eventBus.emit('ai:insight', best) → bus.emit('ai:insight', best)
Step 3: Migrate api/use-query.ts
  - Change import + rename eventBus → bus
Step 4: Migrate api/use-realtime-sync.ts
  - Change import + rename eventBus → bus
Step 5: Verify all 3 hooks compile and emit correctly
Step 6: Remove api/event-bus.ts (now zero consumers)
Step 7: Full test suite + TypeScript compile + build
```

6. Rollback Sequence

```
Step 1: git revert <commit>
Step 2: Verify all 3 event buses restored
Step 3: Verify all 3 hooks still use eventBus
Step 4: Full test suite + TS compile
```

**Rollback confidence:** HIGH — each step is independently reversible. No schema, no data, no API.

7. Verification Plan

| Test                                    | Method          | Expected                             |
|-----------------------------------------|-----------------|--------------------------------------|
| lib/event-bus.ts removed                | File check      | Gone                                 |
| api/event-bus.ts removed                | File check      | Gone                                 |
| No references to eventBus (legacy name) | grep            | 0 occurrences                        |
| 3 hooks use bus.emit instead            | grep            | 3 hooks use bus.emit                 |
| TypeScript compile                      | tsc -b --noEmit | Exit 0                               |
| Frontend build                          | vite build      | Exit 0                               |
| Test suite                              | pytest          | No regressions                       |
| AI insights still emit                  | Manual check    | Insights appear in AI Presence panel |
| Real-time sync still works              | Manual check    | Delta events sync                    |
| Data queries still work                 | Manual check    | use-query fetches + emits            |

8. Candidate Approaches

**Approach A: Direct migration (RECOMMENDED)** Change import + rename method in 3 hooks. Remove both legacy files. - + Simplest: 3 files edited, 2 files removed - + No new abstractions - + All 3 hooks already import canonical bus - – Requires renaming ~5 `eventBus.emit` calls to `bus.emit`

**Approach B: Re-export legacy from canonical** Keep `api/event-bus.ts` as a re-export of `runtimes/event-bus.ts`. - – Creates compatibility layer (forbidden by CEP authorization) - – Keeps dead path alive - – REJECTED: violates "no temporary compatibility layer"

**Approach C: Separate adapter** Create an adapter that maps legacy API to canonical. - – Violates Simplicity Rule §4 - – REJECTED: unnecessary abstraction

9. Risk Assessment

| Risk                              | Severity | Mitigation                                                                        |
|-----------------------------------|----------|-----------------------------------------------------------------------------------|
| rename bug in 3 hooks             | LOW      | 3 files, ~5 method calls total. Each change is import rename + method rename      |
| use-query's eventBus.on() handler | LOW      | Migrate <code>eventBus.on(...)</code> → <code>bus.on(...)</code> — same signature |
| Runtime regression                | LOW      | 3 hooks are independent of runtime orchestration. They just emit/receive events.  |
| Build failure                     | LOW      | tsc catches all type errors                                                       |

**Overall risk: LOWEST.** 3 files changed, 2 files removed, ~5 method calls renamed. All three hooks already import the canonical bus.

10. Experience Impact

**Founder experience: No visible change.** Event bus is internal infrastructure. The founder will not perceive the difference. AI insights, real-time sync, and data queries continue to work identically.

11. Discovery Exit Criteria

| Criterion                     | Status                               |
|-------------------------------|--------------------------------------|
| Canonical ownership confirmed | ✓ <code>runtimes/event-bus.ts</code> |
| Every consumer identified     | ✓ Legacy: 3. Dead: 0. Canonical: 14. |

|                                       |                                  |
|---------------------------------------|----------------------------------|
| Every provider identified             | ✓ All 3 event bus files analyzed |
| Migration order known                 | ✓ 7 steps                        |
| Rollback known                        | ✓ Single git revert              |
| Verification defined                  | ✓ 9 test cases                   |
| No unknown architectural dependencies | ✓ All traced within scope        |

12. Success Evidence

| # | Evidence                              | Measure                                                             |
|---|---------------------------------------|---------------------------------------------------------------------|
| 1 | Canonical Ownership Law §10 satisfied | 3 event buses → 1                                                   |
| 2 | Simplicity Rule §4 satisfied          | 2 files removed                                                     |
| 3 | 3 legacy consumers migrated           | All use <code>bus.emit</code> instead of <code>eventBus.emit</code> |
| 4 | TypeScript compiles                   | Exit 0                                                              |
| 5 | Build succeeds                        | Exit 0                                                              |
| 6 | No regressions                        | 321 tests pass                                                      |
| 7 | Repository simplified                 | −2 files, −2 ownership points                                       |

13. Remaining Unknowns

No remaining architectural unknowns within CEP-003 scope.

All three event buses traced, all consumers identified, migration order defined, rollback validated.

CDR-003 produced by SHUNYA Constitutional Chief Architect Conforms to: CAS-01, LX-06D-R2, LX-06D-R3, EX-02 (infrastructure topology, evidence boundary) Evidence levels: Proven (lib/event-bus.ts dead), Supported (3 legacy consumers identified), Inferred (migration pattern)